

Project 27: Graph Neural Networks for Graph Classification

1. Background and Motivation

Many real-world objects are naturally represented as graphs: molecules (atoms as nodes, bonds as edges), social networks, program ASTs, and interaction graphs. Graph classification aims to predict a label for an entire graph (e.g., toxicity of a molecule, category of a protein graph). Graph Neural Networks (GNNs) extend deep learning to graph-structured data via message passing, enabling learned representations that capture both node attributes and topology. This project focuses on building an end-to-end graph classification pipeline and systematically analyzing architectural choices and their impact on accuracy and efficiency.

2. Problem Description

The primary goal is to train and evaluate GNNs for graph-level prediction, and to perform targeted ablations to understand what matters most.

a) Dataset:

- (recommended, lightweight): a TUDataset benchmark (e.g., MUTAG / PROTEINS / IMDB-BINARY).
- (more realistic): OGB graph property prediction (recommended: ogbg-molhiv).

b) Graph Classification Model Implementation:

- **GCN** or **GraphSAGE** + global pooling (mean/sum/max) + MLP classifier.
- **GIN (Graph Isomorphism Network)** is recommended for a strong, well-established baseline.
- (optimal) If using molecules (ogbg-molhiv), you may optionally use standard atom/bond encodings (must document feature choices).

c) Analysis and Ablations:

- **Pooling Ablation:** compare global mean vs. sum vs. max pooling (or attention pooling).
- **Depth/Capacity Ablation:** compare different numbers of GNN layers (e.g., 2/3/5) and hidden sizes.
- **Feature Ablation:** node features vs. no node features (if applicable), or different feature sets (for molecules).
- **Regularization:** dropout / batch norm / residual connections, and impact on overfitting.

3. Expected Deliverables

Students working in groups (ideally 3 members) are expected to:

- produce a project report,
- deliver a group presentation,
- submit well-commented source code (training + evaluation scripts).

4. Suggested Resources and References

- [1] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*. <https://arxiv.org/pdf/1609.02907>
- [2] Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How Powerful are Graph Neural Networks? (GIN). In *ICLR*. <https://arxiv.org/pdf/1810.00826>
- [3] Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., & Leskovec, J. (2020). Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *NeurIPS*. <https://arxiv.org/pdf/2005.00687>
- [4] Morris, C., et al. (2020). TUDataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint*. <https://arxiv.org/pdf/2007.08663>
- [5] PyTorch Geometric (PyG): Graph Neural Network library and examples. https://github.com/pyg-team/pytorch_geometric
- [6] Open Graph Benchmark (OGB): official repo (datasets, evaluators, baselines). <https://github.com/snap-stanford/ogb>